

```

import { useState } from "react";

const TABS = ["Overview", "Checklist", "Budget", "Vendors", "Timeline"];

const defaultChecklist = [
  { id: 1, category: "12+ Months", task: "Set your budget", done: false },
  { id: 2, category: "12+ Months", task: "Choose a wedding date", done: false },
  { id: 3, category: "12+ Months", task: "Book your venue", done: false },
  { id: 4, category: "12+ Months", task: "Hire a wedding planner (optional)", done: false },
  { id: 5, category: "12+ Months", task: "Start guest list", done: false },
  { id: 6, category: "9-12 Months", task: "Book photographer & videographer", done: false },
  { id: 7, category: "9-12 Months", task: "Book caterer", done: false },
  { id: 8, category: "9-12 Months", task: "Choose wedding party", done: false },
  { id: 9, category: "9-12 Months", task: "Send save-the-dates", done: false },
  { id: 10, category: "6-9 Months", task: "Book florist", done: false },
  { id: 11, category: "6-9 Months", task: "Book band or DJ", done: false },
  { id: 12, category: "6-9 Months", task: "Choose & order wedding dress/attire", done: false },
  { id: 13, category: "6-9 Months", task: "Book officiant", done: false },
  { id: 14, category: "4-6 Months", task: "Send invitations", done: false },
  { id: 15, category: "4-6 Months", task: "Plan honeymoon", done: false },
  { id: 16, category: "4-6 Months", task: "Order wedding cake", done: false },
  { id: 17, category: "4-6 Months", task: "Register for gifts", done: false },
  { id: 18, category: "1-3 Months", task: "Final dress fitting", done: false },
  { id: 19, category: "1-3 Months", task: "Confirm all vendors", done: false },
  { id: 20, category: "1-3 Months", task: "Create seating chart", done: false },
  { id: 21, category: "1 Month", task: "Final guest count to caterer", done: false },
  { id: 22, category: "1 Month", task: "Prepare payments for vendors", done: false },
  { id: 23, category: "1 Week", task: "Rehearsal dinner", done: false },
  { id: 24, category: "1 Week", task: "Pick up rings & attire", done: false },
  { id: 25, category: "1 Week", task: "Pack for honeymoon", done: false },
];

const defaultBudget = [
  { id: 1, category: "Venue", estimated: 8000, actual: 0 },
  { id: 2, category: "Catering", estimated: 6000, actual: 0 },
  { id: 3, category: "Photography", estimated: 3000, actual: 0 },
  { id: 4, category: "Videography", estimated: 2000, actual: 0 },
  { id: 5, category: "Florals", estimated: 2000, actual: 0 },
  { id: 6, category: "Music / DJ", estimated: 1500, actual: 0 },
  { id: 7, category: "Attire & Accessories", estimated: 3000, actual: 0 },
  { id: 8, category: "Wedding Cake", estimated: 600, actual: 0 },
  { id: 9, category: "Officiant", estimated: 400, actual: 0 },
  { id: 10, category: "Invitations & Stationery", estimated: 500, actual: 0 },
  { id: 11, category: "Transportation", estimated: 800, actual: 0 },
];

```

```

    { id: 12, category: "Hair & Makeup", estimated: 800, actual: 0 },
    { id: 13, category: "Honeymoon", estimated: 4000, actual: 0 },
    { id: 14, category: "Miscellaneous", estimated: 1000, actual: 0 },
  ];

const defaultVendors = [
  { id: 1, category: "Venue", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 2, category: "Photographer", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 3, category: "Videographer", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 4, category: "Caterer", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 5, category: "Florist", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 6, category: "DJ / Band", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 7, category: "Officiant", name: "", contact: "", phone: "", email: "", status: "" },
  { id: 8, category: "Hair & Makeup", name: "", contact: "", phone: "", email: "", status: "" },
];

const defaultTimeline = [
  { id: 1, time: "8:00 AM", event: "Hair & Makeup begins", notes: "" },
  { id: 2, time: "12:00 PM", event: "Bridal party gets dressed", notes: "" },
  { id: 3, time: "1:00 PM", event: "First look / photos", notes: "" },
  { id: 4, time: "2:30 PM", event: "Guests arrive", notes: "" },
  { id: 5, time: "3:00 PM", event: "Ceremony begins", notes: "" },
  { id: 6, time: "3:30 PM", event: "Cocktail hour", notes: "" },
  { id: 7, time: "5:00 PM", event: "Reception doors open", notes: "" },
  { id: 8, time: "5:30 PM", event: "Grand entrance & first dance", notes: "" },
  { id: 9, time: "6:00 PM", event: "Dinner service", notes: "" },
  { id: 10, time: "7:00 PM", event: "Toasts & speeches", notes: "" },
  { id: 11, time: "7:30 PM", event: "Cake cutting", notes: "" },
  { id: 12, time: "8:00 PM", event: "Open dancing", notes: "" },
  { id: 13, time: "10:00 PM", event: "Last dance & farewell", notes: "" },
];

const statusColors = {
  "Not Booked": "#e2e8f0",
  "Inquired": "#fef3c7",
  "Booked": "#d1fae5",
  "Paid": "#b2dfdb",
};

const fmt = (n) => `${Number(n || 0).toLocaleString()}`;

export default function WeddingPlanner() {
  const [tab, setTab] = useState("Overview");
  const [coupleName, setCoupleName] = useState("");
  const [weddingDate, setWeddingDate] = useState("");
  const [venue, setVenue] = useState("");
  const [guestCount, setGuestCount] = useState("");

```

```

const [checklist, setChecklist] = useState(defaultChecklist);
const [budget, setBudget] = useState(defaultBudget);
const [vendors, setVendors] = useState(defaultVendors);
const [timeline, setTimeline] = useState(defaultTimeline);
const [editingVendor, setEditingVendor] = useState(null);
const [newTask, setNewTask] = useState("");
const [newBudgetItem, setNewBudgetItem] = useState({ category: "", estimated: "" });
const [newTimeEvent, setNewTimeEvent] = useState({ time: "", event: "" });
const [totalBudget, setTotalBudget] = useState(30000);

const daysUntil = weddingDate
  ? Math.max(0, Math.ceil((new Date(weddingDate) - new Date()) / 86400000))
  : null;

const checklistDone = checklist.filter((t) => t.done).length;
const budgetEstimated = budget.reduce((s, b) => s + Number(b.estimated || 0), 0);
const budgetActual = budget.reduce((s, b) => s + Number(b.actual || 0), 0);
const checklistGroups = checklist.reduce((acc, t) => {
  acc[t.category] = acc[t.category] || [];
  acc[t.category].push(t);
  return acc;
}, {});

const styles = {
  app: {
    fontFamily: "'DM Sans', 'Helvetica Neue', sans-serif",
    minHeight: "100vh",
    background: "#fafaf9",
    color: "#1c1917",
  },
  header: {
    background: "#fff",
    borderBottom: "1px solid #e7e5e4",
    padding: "0 32px",
  },
  headerTop: {
    display: "flex",
    alignItems: "center",
    justifyContent: "space-between",
    padding: "20px 0 0",
    gap: 16,
    flexWrap: "wrap",
  },
  logo: {
    fontSize: 13,
    fontWeight: 600,
    letterSpacing: "0.12em",
  },

```

```

      textTransform: "uppercase",
      color: "#b8962e",
    },
    coupleInput: {
      border: "none",
      outline: "none",
      fontSize: 22,
      fontWeight: 700,
      color: "#b8962e",
      background: "transparent",
      fontFamily: "'DM Sans', sans-serif",
      minWidth: 200,
    },
    },
    dateBadge: {
      display: "flex",
      flexDirection: "column",
      alignItems: "flex-end",
      gap: 2,
    },
    },
    dateLabel: { fontSize: 11, color: "#a8a29e", letterSpacing: "0.08em", textTra
    dateInput: {
      border: "1px solid #e7e5e4",
      borderRadius: 8,
      padding: "6px 10px",
      fontSize: 13,
      fontFamily: "'DM Sans', sans-serif",
      color: "#1c1917",
      background: "#fff",
    },
    },
    daysTag: {
      background: "#1c1917",
      color: "#fff",
      borderRadius: 20,
      padding: "4px 12px",
      fontSize: 12,
      fontWeight: 600,
      marginLeft: 8,
    },
    },
    tabs: {
      display: "flex",
      gap: 0,
      marginTop: 16,
    },
    },
    tab: (active) => ({
      padding: "10px 20px",
      fontSize: 13,
      fontWeight: active ? 600 : 400,

```

```

    color: active ? "#1c1917" : "#78716c",
    borderBottom: active ? "2px solid #1c1917" : "2px solid transparent",
    cursor: "pointer",
    background: "none",
    border: "none",
    borderBottom: active ? "2px solid #1c1917" : "2px solid transparent",
    transition: "all 0.15s",
  )),
main: {
  maxWidth: 900,
  margin: "0 auto",
  padding: "32px 24px",
},
card: {
  background: "#fff",
  border: "1px solid #e7e5e4",
  borderRadius: 12,
  padding: 24,
  marginBottom: 20,
},
sectionTitle: {
  fontSize: 11,
  fontWeight: 700,
  letterSpacing: "0.1em",
  textTransform: "uppercase",
  color: "#a8a29e",
  marginBottom: 16,
},
statsGrid: {
  display: "grid",
  gridTemplateColumns: "repeat(auto-fit, minmax(160px, 1fr))",
  gap: 16,
  marginBottom: 20,
},
statCard: {
  background: "#fff",
  border: "1px solid #e7e5e4",
  borderRadius: 12,
  padding: "20px 24px",
},
statValue: { fontSize: 28, fontWeight: 700, color: "#1c1917", lineHeight: 1 }
statLabel: { fontSize: 12, color: "#a8a29e", marginTop: 6 },
progressBar: (pct, color = "#1c1917") => ({
  height: 6,
  borderRadius: 3,
  background: "#f5f5f4",
  overflow: "hidden",

```

```

    marginTop: 8,
    position: "relative",
  )),
  progressFill: (pct, color = "#1c1917") => ({
    height: "100%",
    width: `${Math.min(100, pct)}%`,
    background: color,
    borderRadius: 3,
    transition: "width 0.4s ease",
  )),
  row: {
    display: "flex",
    alignItems: "center",
    gap: 12,
    padding: "10px 0",
    borderBottom: "1px solid #f5f5f4",
  },
  checkbox: {
    width: 18,
    height: 18,
    accentColor: "#1c1917",
    cursor: "pointer",
    flexShrink: 0,
  },
  taskText: (done) => ({
    fontSize: 14,
    color: done ? "#a8a29e" : "#1c1917",
    textDecoration: done ? "line-through" : "none",
    flex: 1,
  )),
  catHeader: {
    fontSize: 11,
    fontWeight: 700,
    letterSpacing: "0.1em",
    textTransform: "uppercase",
    color: "#a8a29e",
    padding: "16px 0 8px",
    marginTop: 8,
  },
  input: {
    border: "1px solid #e7e5e4",
    borderRadius: 8,
    padding: "8px 12px",
    fontSize: 13,
    fontFamily: "'DM Sans', sans-serif",
    color: "#1c1917",
    outline: "none",
  },

```

```

        background: "#fff",
    },
    btn: {
        background: "#1c1917",
        color: "#fff",
        border: "none",
        borderRadius: 8,
        padding: "8px 16px",
        fontSize: 13,
        fontWeight: 600,
        cursor: "pointer",
        fontFamily: "'DM Sans', sans-serif",
    },
    btnOutline: {
        background: "#fff",
        color: "#1c1917",
        border: "1px solid #e7e5e4",
        borderRadius: 8,
        padding: "6px 12px",
        fontSize: 12,
        fontWeight: 600,
        cursor: "pointer",
        fontFamily: "'DM Sans', sans-serif",
    },
    table: { width: "100%", borderCollapse: "collapse" },
    th: {
        fontSize: 11,
        fontWeight: 700,
        letterSpacing: "0.08em",
        textTransform: "uppercase",
        color: "#a8a29e",
        textAlign: "left",
        padding: "8px 12px 8px 0",
        borderBottom: "1px solid #e7e5e4",
    },
    td: {
        padding: "12px 12px 12px 0",
        fontSize: 14,
        borderBottom: "1px solid #f5f5f4",
        verticalAlign: "middle",
    },
    numInput: {
        border: "1px solid #e7e5e4",
        borderRadius: 6,
        padding: "5px 8px",
        fontSize: 13,
        width: 90,
    }

```

```

    fontFamily: "'DM Sans', sans-serif",
    color: "#1c1917",
    outline: "none",
  },
  statusBadge: (s) => ({
    display: "inline-block",
    padding: "3px 10px",
    borderRadius: 20,
    fontSize: 11,
    fontWeight: 600,
    background: statusColors[s] || "#e2e8f0",
    color: "#1c1917",
  })),
  select: {
    border: "1px solid #e7e5e4",
    borderRadius: 6,
    padding: "5px 8px",
    fontSize: 12,
    fontFamily: "'DM Sans', sans-serif",
    color: "#1c1917",
    background: "#fff",
    outline: "none",
  },
  timelineItem: {
    display: "flex",
    gap: 16,
    alignItems: "flex-start",
    padding: "14px 0",
    borderBottom: "1px solid #f5f5f4",
  },
  timeLabel: {
    minWidth: 70,
    fontSize: 12,
    fontWeight: 700,
    color: "#a8a29e",
    paddingTop: 2,
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: "50%",
    background: "#1c1917",
    marginTop: 6,
    flexShrink: 0,
  },
};

```

```

const overviewContent = (
  <>
    <div style={styles.statsGrid}>
      <div style={styles.statCard}>
        <div style={styles.statValue}>{daysUntil !== null ? daysUntil : "-"}</div>
        <div style={styles.statLabel}>Days Until Wedding</div>
      </div>
      <div style={styles.statCard}>
        <div style={styles.statValue}>{checklistDone}/{checklist.length}</div>
        <div style={styles.statLabel}>Tasks Complete</div>
        <div style={styles.progressBar()}>
          <div style={styles.progressFill((checklistDone / checklist.length) *
            </div>
          </div>
        <div style={styles.statCard}>
          <div style={styles.statValue}>{fmt(budgetActual)}</div>
          <div style={styles.statLabel}>Spent of {fmt(budgetEstimated)} estimated
          <div style={styles.progressBar()}>
            <div style={styles.progressFill((budgetActual / budgetEstimated) * 10
              </div>
            </div>
          <div style={styles.statCard}>
            <div style={styles.statValue}>{vendors.filter(v => v.status === "Booked
              <div style={styles.statLabel}>Vendors Booked</div>
            </div>
          </div>

    <div style={styles.card}>
      <div style={styles.sectionTitle}>Wedding Details</div>
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 16 }}
        {[
          { label: "Couple / Names", val: coupleName, set: setCoupleName, place
            { label: "Venue", val: venue, set: setVenue, placeholder: "Venue name
            { label: "Guest Count", val: guestCount, set: setGuestCount, placehol
            { label: "Total Budget", val: totalBudget, set: setTotalBudget, place
        ].map((f) => (
          <div key={f.label}>
            <div style={{ fontSize: 11, color: "#a8a29e", fontWeight: 700, text
              <input
                style={{ ...styles.input, width: "100%", boxSizing: "border-box"
                  value={f.val}
                  onChange={e => f.set(e.target.value)}
                  placeholder={f.placeholder}
                  type={f.type || "text"}
              </div>
            </div>
          </div>
        ))}

```

```

    </div>
  </div>

  <div style={styles.card}>
    <div style={styles.sectionTitle}>Budget Remaining</div>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
      <span style={{ fontSize: 32, fontWeight: 700 }}>{fmt(totalBudget - budgetActual)}</span>
      <span style={{ fontSize: 13, color: "#a8a29e" }}>of {fmt(totalBudget)}</span>
    </div>
    <div style={styles.progressBar}>
      <div style={styles.progressFill(((totalBudget - budgetActual) / totalBudget) * 100)}></div>
    </div>
  </div>
</>
);

const checklistContent = (
  <div style={styles.card}>
    <div style={styles.sectionTitle}>Wedding Checklist</div>
    {Object.entries(checklistGroups).map(([cat, tasks]) => (
      <div key={cat}>
        <div style={styles.catHeader}>{cat}</div>
        {tasks.map(t => (
          <div key={t.id} style={styles.row}>
            <input
              type="checkbox"
              checked={t.done}
              style={styles.checkbox}
              onChange={() => setChecklist(prev => prev.map(x => x.id !== t.id ? x : { ...x, done: !t.done })}
            />
            <span style={styles.taskText(t.done)}>{t.task}</span>
            <button
              style={{ ...styles.btnOutline, padding: "3px 8px", fontSize: 11, color: "#a8a29e" }}
              onClick={() => setChecklist(prev => prev.filter(x => x.id !== t.id)}
            >x</button>
          </div>
        ))}
      </div>
    ))}
  </div>
)}

<div style={{ display: "flex", gap: 8, marginTop: 20 }}>
  <input
    style={{ ...styles.input, flex: 1 }}
    placeholder="Add a task..."
    value={newTask}
    onChange={e => setNewTask(e.target.value)}
    onKeyDown={e => {
      if (e.key === "Enter" && newTask.trim()) {

```

```

        setChecklist(prev => [...prev, { id: Date.now(), category: "Custom"
        setNewTask("");
    }
  }}
/>
<button style={styles.btn} onClick={() => {
  if (newTask.trim()) {
    setChecklist(prev => [...prev, { id: Date.now(), category: "Custom",
    setNewTask("");
  }
}}>Add</button>
</div>
</div>
);

const budgetContent = (
  <div style={styles.card}>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
    <div style={styles.sectionTitle}>Budget Tracker</div>
    <div style={{ textAlign: "right" }}>
      <div style={{ fontSize: 12, color: "#a8a29e" }}>Remaining</div>
      <div style={{ fontSize: 20, fontWeight: 700, color: budgetEstimated > t
      {fmt(totalBudget - budgetActual)}
    </div>
  </div>
</div>
<table style={styles.table}>
  <thead>
    <tr>
      {[ "Category", "Estimated", "Actual", "Difference", "" ].map(h => (
        <th key={h} style={styles.th}>{h}</th>
      ))}
    </tr>
  </thead>
  <tbody>
    {budget.map(b => {
      const diff = Number(b.estimated || 0) - Number(b.actual || 0);
      return (
        <tr key={b.id}>
          <td style={styles.td}><strong>{b.category}</strong></td>
          <td style={styles.td}>
            <input
              style={styles.numInput}
              type="number"
              value={b.estimated}
              onChange={e => setBudget(prev => prev.map(x => x.id === b.id
            />

```

```

        </td>
        <td style={styles.td}>
          <input
            style={styles.numInput}
            type="number"
            value={b.actual}
            onChange={e => setBudget(prev => prev.map(x => x.id === b.id
              />
            </td>
            <td style={{ ...styles.td, color: diff >= 0 ? "#16a34a" : "#ef4444"
              {diff >= 0 ? "+" : ""}{fmt(diff)}}
            </td>
            <td style={styles.td}>
              <button
                style={{ ...styles.btnOutline, padding: "3px 8px", fontSize:
                  onClick={() => setBudget(prev => prev.filter(x => x.id !== b.
                >x</button>
              </td>
            </tr>
          );
        }}}
      <tr>
        <td style={{ ...styles.td, fontWeight: 700 }}>Total</td>
        <td style={{ ...styles.td, fontWeight: 700 }}>{fmt(budgetEstimated)}<
        <td style={{ ...styles.td, fontWeight: 700 }}>{fmt(budgetActual)}</td>
        <td style={{ ...styles.td, fontWeight: 700, color: budgetEstimated >=
          {fmt(budgetEstimated - budgetActual)}
        </td>
        <td />
      </tr>
    </tbody>
  </table>
  <div style={{ display: "flex", gap: 8, marginTop: 20, flexWrap: "wrap" }}>
    <input
      style={{ ...styles.input, flex: 1, minWidth: 120 }}
      placeholder="Category name"
      value={newBudgetItem.category}
      onChange={e => setNewBudgetItem(p => ({ ...p, category: e.target.value
    />
    <input
      style={{ ...styles.input, width: 110 }}
      placeholder="Estimated $"
      type="number"
      value={newBudgetItem.estimated}
      onChange={e => setNewBudgetItem(p => ({ ...p, estimated: e.target.value
    />
    <button style={styles.btn} onClick={() => {

```

```

        if (newBudgetItem.category.trim()) {
          setBudget(prev => [...prev, { id: Date.now(), category: newBudgetItem
            setNewBudgetItem({ category: "", estimated: "" }));
        }
      }}>Add Item</button>
    </div>
  </div>
);

const vendorContent = (
  <div style={styles.card}>
    <div style={styles.sectionTitle}>Vendor Tracker</div>
    <table style={styles.table}>
      <thead>
        <tr>
          {[ "Category", "Name", "Contact / Phone", "Status", "Notes", "" ].map(h
            <th key={h} style={styles.th}>{h}</th>
          )]}
        </tr>
      </thead>
      <tbody>
        {vendors.map(v => (
          <tr key={v.id}>
            <td style={{ ...styles.td, fontWeight: 600, whiteSpace: "nowrap" }}>
              <td style={styles.td}>
                <input
                  style={{ ...styles.input, width: 120 }}
                  placeholder="Business name"
                  value={v.name}
                  onChange={e => setVendors(prev => prev.map(x => x.id === v.id ?
                    />
                </td>
                <td style={styles.td}>
                  <input
                    style={{ ...styles.input, width: 130 }}
                    placeholder="Phone or email"
                    value={v.phone || v.email}
                    onChange={e => setVendors(prev => prev.map(x => x.id === v.id ?
                      />
                </td>
                <td style={styles.td}>
                  <select
                    style={styles.select}
                    value={v.status}
                    onChange={e => setVendors(prev => prev.map(x => x.id === v.id ?
                      >
                    {Object.keys(statusColors).map(s => <option key={s}>{s}</option>

```

```

        </select>
      </td>
      <td style={styles.td}>
        <input
          style={{ ...styles.input, width: 140 }}
          placeholder="Notes..."
          value={v.notes}
          onChange={e => setVendors(prev => prev.map(x => x.id === v.id ?
            />
          </td>
        <td style={styles.td}>
          <button
            style={{ ...styles.btnOutline, padding: "3px 8px", fontSize: 11
            onClick={() => setVendors(prev => prev.filter(x => x.id !== v.i
            >x</button>
          </td>
        </tr>
      )}}
    </tbody>
  </table>
  <button
    style={{ ...styles.btn, marginTop: 16, background: "#f5f5f4", color: "#1c
    onClick={() => setVendors(prev => [...prev, { id: Date.now(), category: "
    >+ Add Vendor</button>
  </div>
);

const timelineContent = (
  <div style={styles.card}>
    <div style={styles.sectionTitle}>Wedding Day Timeline</div>
    {timeline.map(t => (
      <div key={t.id} style={styles.timelineItem}>
        <div style={styles.dot} />
        <div style={styles.timeLabel}>{t.time}</div>
        <input
          style={{ ...styles.input, flex: 1 }}
          value={t.event}
          onChange={e => setTimeline(prev => prev.map(x => x.id === t.id ? { ..
        />
        <input
          style={{ ...styles.input, width: 160 }}
          placeholder="Notes..."
          value={t.notes}
          onChange={e => setTimeline(prev => prev.map(x => x.id === t.id ? { ..
        />
        <button
          style={{ ...styles.btnOutline, padding: "3px 8px", fontSize: 11, colo

```

```

        onClick={() => setTimeline(prev => prev.filter(x => x.id !== t.id))}
      >x</button>
    </div>
  )))
<div style={{ display: "flex", gap: 8, marginTop: 20, flexWrap: "wrap" }}>
  <input
    style={{ ...styles.input, width: 90 }}
    placeholder="Time"
    value={newTimeEvent.time}
    onChange={e => setNewTimeEvent(p => ({ ...p, time: e.target.value })))}
  />
  <input
    style={{ ...styles.input, flex: 1, minWidth: 160 }}
    placeholder="Event description"
    value={newTimeEvent.event}
    onChange={e => setNewTimeEvent(p => ({ ...p, event: e.target.value })))}
  />
  <button style={styles.btn} onClick={() => {
    if (newTimeEvent.event.trim()) {
      setTimeline(prev => [...prev, { id: Date.now(), time: newTimeEvent.time, event: newTimeEvent.event }]);
      setNewTimeEvent({ time: "", event: "" });
    }
  }}>Add</button>
</div>
</div>
);

const contentMap = {
  Overview: overviewContent,
  Checklist: checklistContent,
  Budget: budgetContent,
  Vendors: vendorContent,
  Timeline: timelineContent,
};

return (
  <div style={styles.app}>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600" rel="stylesheet">
    <div style={styles.header}>
      <div style={styles.headerTop}>
        <div>
          <div style={styles.logo}>Wedding Planner</div>
          <input
            style={styles.coupleInput}
            placeholder="Enter couple's names..."
            value={coupleName}
            onChange={e => setCoupleName(e.target.value)}
          />
        </div>
      </div>
    </div>
  </div>
);

```

```

        />
    </div>
    <div style={styles.dateBadge}>
      <span style={styles.dateLabel}>Wedding Date</span>
      <div style={{ display: "flex", alignItems: "center", gap: 8 }}>
        <input
          type="date"
          style={styles.dateInput}
          value={weddingDate}
          onChange={e => setWeddingDate(e.target.value)}
        />
        {daysUntil !== null && (
          <span style={styles.daysTag}>{daysUntil}d</span>
        )}
      </div>
    </div>
  </div>
  <div style={styles.tabs}>
    {TABS.map(t => (
      <button key={t} style={styles.tab(tab === t)} onClick={() => setTab(t)}
    ))}
  </div>
</div>
<div style={styles.main}>
  {contentMap[tab]}
</div>
</div>
);
}

```